



Using Security APIs and Role Escalation

Version 2026.1
2026-04-20

Using Security APIs and Role Escalation
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1
Copyright © 2026 InterSystems Corporation
All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

- Using Security APIs and Role Escalation..... 1
 - 1 Acquiring Privileges 1
 - 2 Role Escalation 1
 - 2.1 Macros 3
 - 3 Caller Verification 3
 - 4 Querying Security Tables 5
 - 4.1 Query API 5
 - 4.2 Embedded and Dynamic SQL 7

Using Security APIs and Role Escalation

Starting with InterSystems IRIS 2025.2, all security APIs require the following privileges:

- **%DB_IRISSYS:R**
- **%Admin_Secure:U**

The OAuth2 classes also require one of the following, depending on the class and operation:

- **%Admin_OAuth2_Client:U** — Creating and using InterSystems IRIS as a client or resource server.
- **%Admin_OAuth2_Server:U** — Configuring and using InterSystems IRIS as an authorization server.
- **%Admin_OAuth2_Registration:U** — Revoking access tokens and registering clients for an InterSystems IRIS authorization server.

If a security API is called without these privileges, an **Access Denied** error is returned to the caller, and the attempt is recorded in the audit log as a **%System/%Security/AccessDenied** event.

1 Acquiring Privileges

You can get the required privileges in the following ways:

- Role assignment — Owning a role that has the privilege. The following is a list of each privilege and the predefined roles that have them:
 - **%DB_IRISSYS:R** — %DB_IRISSYS_READ, %Manager
 - **%Admin_Secure:U** — %Admin_Secure, %SecurityAdministrator
 - **%Admin_OAuth2_*:U** — %SecurityAdministrator
- Role escalation for IRISSYS routines — Executing a % routine or class which physically resides in the IRISSYS database and sets **\$ROLES** directly.
- [Application or matching roles](#) escalation
- **%All** role — Having the **%All** role gives this (and all other) privileges.

The second method, role escalation, is the most versatile and is the primary subject of this article.

2 Role Escalation

A process can modify its privileges through role escalation by [adding a role to its \\$ROLES variable](#) (for example, the predefined **%DB_IRISSYS_READ** and **%Admin_Secure** roles) if any of the following are true:

- The method or routine resides in the **IRISSYS** database.
- The method or routine is mapped to the **IRISSYS** database.

As a best practice, you should also use **NEW \$ROLES** to ensure that the role escalation only lasts until the end of the method or routine.

For example, a method can add use the role **NewRole** until the end of its call with the following:

ObjectScript

```
NEW $ROLES
SET $ROLES=$ROLES_",NewRole"
```

You can use this to escalate roles for the relevant security APIs. For example, to escalate to the predefined role **%Admin_Secure**:

ObjectScript

```
NEW $ROLES
SET $ROLES=$ROLES_",%Admin_Secure"
```

In this example, the routine **SECURITYAPI** is in the **USER** namespace and resides in the **USER** database (IRISinstallDir\mgr\user\). **AddRole()** attempts to modify the **\$ROLES** value to escalate to **%Admin_Secure**:

ObjectScript

```
AddRole() public {
    WRITE !,"$ROLES="_$ROLES
    SET $ROLES=$ROLES_",%Admin_Secure"
    WRITE !,"$ROLES="_$ROLES
    QUIT
}
```

However, executing this routine results in a **PROTECT** error because the routine does not reside in and is not mapped to the **IRISSYS** database, which are the prerequisites for **SET \$ROLES**:

```
USER>DO AddRole^SECURITYAPI
$ROLES=%Developer,%DB_USER
SET $ROLES=$ROLES_",%Admin_Secure"
^
<PROTECT>SECURITYAPI+5^SECURITYAPI
USER 2d0>
```

To fix this, you can either move or map the routine to the **IRISSYS** database. Classes and routines that start with **%Z** or **%z** are **automatically mapped** to the **IRISSYS** database.

This example uses the mapping method, changing the name of the routine to **%ZSECURITYAPI**, which maps it to the **IRISSYS** database. This allows the process to run **%** routines and classes:

```
USER>DO AddRole^%ZSECURITYAPI
$ROLES=%Developer,%DB_USER
$ROLES=%Developer,%Admin_Secure,%DB_USER
USER>WRITE $ROLES
%Developer,%Admin_Secure,%DB_USER
```

For completeness, this is the same routine written in a **%Z** class:

Class Definition

```
Class %ZUSER.Security [ Abstract ]
{
ClassMethod AddRole()
{
    WRITE !,"$ROLES="_$roles
    SET $ROLES=$ROLES_",%Admin_Secure"
    WRITE !,"$ROLES="_$ROLES
    QUIT
}
}
```

Again, the class name starts with %Z, so it's automatically mapped to the **IRISSYS** database:

```
USER>DO ##Class(%ZUSER.Security).AddRole()
$ROLES=%Developer,%DB_USER
$ROLES=%Developer,%Admin_Secure,%DB_USER
USER>WRITE $ROLES
%Developer,%Admin_Secure,%DB_USER
```

2.1 Macros

To make role escalation more convenient, you can define [macros in an include file](#). In this example, the include file is called %ZUSERSecurity.inc:

ObjectScript

```
; Escalates to %DB_IRISSYS_READ, giving the caller %IRISSYS:R
#Define AddBIRISSYSRead NEW $ROLES SET $ROLES=$ROLES_", %DB_IRISSYS_READ"
; Escalates to %Admin_Secure, giving the caller %Admin_Secure:U
#Define AddAdminSecure NEW $ROLES SET $ROLES=$ROLES_", %Admin_Secure"
```

Building on the previous example class %ZUSER.Security, you can add the following to make it more idiomatic:

- Add the include file (in this case, %ZUSERSecurity.inc)
- Add **TRY/CATCH** error handling.
- Verify that the caller has the privileges to call the method.
- Restrict which classes and routines can call the method.

3 Caller Verification

By default, any process in the system can call %Z and %Z classes directly. To restrict this, you can add checks to the beginning of the method to verify that the caller fulfills the specified requirements:

- **%SYSTEM.Security.Check()** — Verifies that the calling user has the specified privileges. As a best practice, you should only explicitly check for permissions on user-defined resources; predefined resources can potentially change between InterSystems IRIS versions.
- **%SYSTEM.Process.CallingDatabase()** — Returns the directory name of the calling routine's database.
- **%SYSTEM.Process.CallingRoutine()** — Returns the name of the calling routine.

In general, you should use at least one of these checks for methods and routines that read or modify security data.

For example, you can add the following line to the beginning of a method to verify that the calling user has the user-defined APPSECURITY:U privilege:

ObjectScript

```
if '$SYSTEM.Security.Check("APPSECURITY","USE") QUIT $$$ERROR($$$AccessDenied)
```

Similarly, suppose you're running the class USER.APP in the **USER** namespace and want to call the method **GetUsernameInfo()**:

Class Definition

```
Class USER.APP [ Abstract ]
{
ClassMethod GetUsernameInfo(Username as %String, byRef Properties as %String) As %Status
{
    QUIT ##Class(%ZUSER.Security).GetUsernameInfo(Username, .Properties)
}
}
```

You can modify the method as follows to test the calling database and routine and verify it can only be called from the USER.APP class in the **USER** database:

Class Definition

```
Include (%ZUSERSecurity, %occErrors)
Class %ZUSER.Security [ Abstract ]
{
ClassMethod GetUsernameInfo(Username As %String, ByRef UserInfo As %String) As %Status
{
    TRY {
        #;Calling user must have the APPSECURITY:USE privilege
        IF '$SYSTEM.Security.Check("APPSECURITY","USE") $$$ThrowStatus($$$$ERROR($$$AccessDenied))

        #;Calling class must be USER.APP and physically located in the USER database
        IF ($SYSTEM.Process.CallingRoutine()!="USER.APP.1") ||
        ($SYSTEM.Process.CallingDatabase()!="iris/mgr/user/") $$$ThrowStatus($$$$ERROR($$$AccessDenied))

        #;Required to change to %SYS namespace to use APIs
        $$$AddDBIRISSYSRead

        #;Required to access security APIs
        $$$AddAdminSecure

        #;On method exit, restore to calling namespace
        NEW $Namespace
        SET $Namespace="%SYS"

        $$$ThrowOnError(##Class(Security.Users).Get(Username, .UserInfo))
    } CATCH Exception {
        RETURN Exception.AsStatus()
    }
    QUIT $$$OK
}
}
```

Here, **GetUsernameInfo()** shows that the user Bob is enabled:

```
USER>SET Status=##Class(USER.APP).GetUsernameInfo("Bob", .UserInfo)
USER>WRITE UserInfo("Enabled")
UserInfo("Enabled")=1
```

The following method disables a user. Before performing role escalation with the **\$\$\$AddDBIRISSYSREAD** and **\$\$\$AddAdminSecure** macros, it verifies that the calling user has APPSECURITY:U and that the calling routine/method is either **USER.APP.1** or exists in the **USER** database:

ObjectScript

```
ClassMethod DisableUser(Username As %String) As %Status
{
    TRY {
        IF '$SYSTEM.Security.Check("APPSECURITY","USE") $$$ThrowStatus($$$$ERROR($$$AccessDenied))
        IF ($SYSTEM.Process.CallingRoutine()!="USER.APP.1") ||
        ($SYSTEM.Process.CallingDatabase()!="iris/mgr/user/") $$$ThrowStatus($$$$ERROR($$$AccessDenied))
        $$$AddDBIRISSYSRead
        $$$AddAdminSecure
        NEW $Namespace
        SET $Namespace="%SYS"
        SET Properties("Enabled")=0
        $$$ThrowOnError(##Class(Security.Users).Modify(Username, .Properties))
    } CATCH Exception {
        RETURN Exception.AsStatus()
    }
    QUIT $$$OK
}
```


To disable the user Bob:

ObjectScript

```
USER>SET Status=##Class(USER.APP).DisableUser("Bob")
USER>WRITE Status
1
USER>DO ##Class(%ZUSER.Security).GetUsernameInfo("Bob",.UserInfo)
USER>WRITE UserInfo("Enabled")
0
```

This next example creates a new role `OAuthRole` which has the following privileges:

- `%Admin_OAuth2_Client:U`
- `%Admin_OAuth2_Server:U`
- `%Admin_OAuth2_Registration:U`

ObjectScript

```
ClassMethod CreateOAuthRole() As %Status
{
    TRY {
        IF '$SYSTEM.Security.Check("APPSECURITY","USE")' $$$ThrowStatus($$$$ERROR($$$$AccessDenied))
        IF ($SYSTEM.Process.CallingRoutine()='USER.APP.1') ||
        ($SYSTEM.Process.CallingDatabase()='iris/mgr/user/') $$$ThrowStatus($$$$ERROR($$$$AccessDenied))
        $$$AddDBIRISSYSRead
        $$$AddAdminSecure
        NEW $Namespace
        SET $Namespace="%SYS"
        $$$ThrowOnError(##Class(Security.Roles).Create("OAuthRole","OAuth privilege role",
"%Admin_OAuth2_Client:Use,%Admin_OAuth2_Server:Use,%Admin_OAuth2_Registration:Use"))
        RETURN $$$OK
    } CATCH Exception {
        RETURN Exception.AsStatus()
    }
}
```

For convenience, you can create a macro to escalate to the new `OAuthRole`:

ObjectScript

```
#Define AddOAuthRole New $Roles Set $Roles=$Roles_",OAuthRole"
```

4 Querying Security Tables

Like globals, without **%ALL**, security tables are restricted and can only be accessed through security APIs.

You can query security tables with the following:

- [Query API](#)
- [Embedded and Dynamic SQL](#)

4.1 Query API

The Query API lets you query security tables with the following:

- Predefined Query methods
- Custom queries with `Security.SQLQuery`

Like all other security APIs and classes, these [require](#) `%DB_IRISSYS:R` and `%Admin_Secure:U`.

The following example uses a predefined Query method. To get a list of roles, instead of querying the `Security.Roles` table, it uses the predefined `Security.Roles_List()` query:

```
SELECT Name,Description FROM Security.Roles_List()
```

The following example runs the `Security.Users_List()` to return the contents of the Enabled column:

ObjectScript

```
ClassMethod GetUsersEnabledState(ByRef Users As %String) As %Status
{
    TRY {
        IF '$SYSTEM.Security.Check("APPSECURITY","USE") $$$ThrowStatus($$ERROR($$$AccessDenied))
        IF ($SYSTEM.Process.CallingRoutine()='="USER.APP.1") ||
        ($SYSTEM.Process.CallingDatabase()='="/iris/mgr/user/") $$$ThrowStatus($$ERROR($$$AccessDenied))
        KILL Users
        $$$AddDBIRISSYSRead
        $$$AddAdminSecure
        NEW $Namespace
        SET $Namespace="%SYS"
        SET stmt = ##class(%SQL.Statement).%New()
        $$$ThrowOnError(stmt.%PrepareClassQuery("Security.Users","List",0))
        SET rs = stmt.%Execute("").%NextResult()
        IF rs.%SQLCODE < 0 $$$ThrowStatus($$ERROR($$$SQLError, rs.%SQLCODE, rs.%Message))
        WHILE rs.%Next() {
            SET Users(rs.Name)=rs.Enabled
        }
        RETURN $$$OK
    } CATCH Exception {
        RETURN Exception.AsStatus()
    }
}
```

If you want to create a custom query instead of using the predefined ones, you can create a class query with class `Security.SQLQuery`. For example:

ObjectScript

```
Query ListEnabled(ROWSPEC = "Name:%String,Enabled:%Boolean") As Security.SQLQuery [ SqlProc ]
{
    SELECT Name,Enabled FROM Security.Users
    ORDER BY Name
}
```

You can then use the custom query in your methods. For example, `GetUsersEnabledStateSQLQuery()` calls `ListEnabled`. Because this query, like all other security APIs, requires elevated privileges, this method and performs checks its callers before escalating to `%DB_IRISSYS_READ` and `%Admin_Secure`:

ObjectScript

```
ClassMethod GetUsersEnabledStateSQLQuery(ByRef Users As %String) As %Status
{
    TRY {
        IF '$SYSTEM.Security.Check("APPSECURITY","USE") $$$ThrowStatus($$ERROR($$$AccessDenied))
        IF ($SYSTEM.Process.CallingRoutine()='="USER.APP.1") ||
        ($SYSTEM.Process.CallingDatabase()='="/iris/mgr/user/") $$$ThrowStatus($$ERROR($$$AccessDenied))
        KILL Users
        $$$AddDBIRISSYSRead
        $$$AddAdminSecure
        NEW $Namespace
        SET $Namespace="%SYS"
        SET stmt = ##class(%SQL.Statement).%New()
        $$$ThrowOnError(stmt.%PrepareClassQuery("%ZUSER.Security","ListEnabled",0))
        SET rs = stmt.%Execute().%NextResult()
        IF rs.%SQLCODE < 0 $$$ThrowStatus($$ERROR($$$SQLError, rs.%SQLCODE, rs.%Message))
        WHILE rs.%Next() {
            SET Users(rs.Name)=rs.Enabled
        }
        RETURN $$$OK
    } CATCH Exception {
```

```

    }
    RETURN Exception.AsStatus()
}

```

4.2 Embedded and Dynamic SQL

While you can query the security database with [Embedded](#) and [Dynamic](#) SQL, this accesses security globals directly (as opposed to the provided APIs); this form of direct access requires **%All**, which is more dangerous and not recommended.

The following examples use a macro to escalate to **%All**:

ObjectScript

```
#define AddAllRole New $Roles Set $Roles=$Roles_",%All"
```

To return the contents of the Enabled column with embedded SQL:

ObjectScript

```

ClassMethod GetUsersEnabledStateEmbeddedSQL(ByRef Users As %String) As %Status
{
    TRY {
        IF '$SYSTEM.Security.Check("APPSECURITY","USE") $$$ThrowStatus($$$$ERROR($$$AccessDenied))
        IF ($SYSTEM.Process.CallingRoutine()'="USER.APP.1") ||
        ($SYSTEM.Process.CallingDatabase()'="/iris/mgr/user/") $$$ThrowStatus($$$$ERROR($$$AccessDenied))
        KILL Users
        $$$AddAllRole
        NEW $Namespace
        SET $Namespace="%SYS"
        &sql(DECLARE C1 CURSOR FOR
            SELECT Name,Enabled INTO :name,:enabled
            FROM Security.Users
            ORDER BY Name)
        &sql(OPEN C1)
        QUIT:(SQLCODE'=0)
        &sql(FETCH C1)
        WHILE (SQLCODE = 0) {
            SET Users(name)=enabled
            &sql(FETCH C1)
        }

        &sql(CLOSE C1)
        RETURN $$$OK
    } CATCH Exception {
        RETURN Exception.AsStatus()
    }
}

```

Similarly, to return the contents of the Enabled column with dynamic SQL:

```

ClassMethod GetUsersEnabledStateDynamicSQL(ByRef Users As %String) As %Status
{
    TRY {
        IF '$SYSTEM.Security.Check("APPSECURITY","USE") $$$ThrowStatus($$$$ERROR($$$AccessDenied))
        IF ($SYSTEM.Process.CallingRoutine()'="USER.APP.1") ||
        ($SYSTEM.Process.CallingDatabase()'="/iris/mgr/user/") $$$ThrowStatus($$$$ERROR($$$AccessDenied))
        KILL Users
        $$$AddAllRole
        NEW $Namespace
        SET $Namespace="%SYS"
        SET query="SELECT * FROM Security.Users"
        SET rs=##class(%SQL.Statement).%ExecDirect(,query)
        IF (rs.%SQLCODE < 0) $$$ThrowStatus($$$$ERROR($$$$SQLError, rs.%SQLCODE, rs.%Message))
        WHILE rs.%Next(.sc) {
            Set Users(rs.%Get("Name"))=rs.%Get("Enabled")
        }
        RETURN $$$OK
    } CATCH Exception {
        RETURN Exception.AsStatus()
    }
}

```

